# A LangGraph-Based Multi-Agent SOC Analyst System

---

## Table of Contents


1. Objective
2. Theory / Background
3. System Architecture
4. System Working and Different Use Cases
5. Pre-requisites and Environment Setup
6. Configuration Steps
7. Agent Design and Roles
8. Orchestration and State Management
9. Data Sources and Tool Integrations
10. Prompts and LLM Configuration
11. Evaluation and Performance Metrics
12. Security and Privacy Considerations
13. Limitations and Future Scope
14. Conclusion
15. Appendices


---

## 1. Objective

### 1.1 Problem Statement

Security Operations Centers (SOCs) face an ever-growing volume of security alerts — often thousands per day — from firewalls, intrusion detection systems, endpoint protection, and SIEM platforms. Analysts must manually triage, enrich, correlate, and respond to each alert, a process that is:

- **Time-consuming**: A single investigation can take 30–60 minutes of manual querying across multiple platforms.
- **Error-prone**: Human fatigue leads to missed indicators, especially during off-hours.
- **Inconsistent**: Different analysts apply different playbooks, leading to variable response quality.
- **Unscalable**: Alert volume grows faster than headcount; many organizations have a 70%+ alert fatigue rate.

### 1.2 Project Goal

This project aims to automate the SOC analyst workflow by deploying a **multi-agent AI system** built on LangGraph. Given a raw security alert (JSON), the system autonomously:

1. **Triages** the alert — extracting IOCs (Indicators of Compromise), mapping to MITRE ATT&CK, and scoring severity.
2. **Enriches** each IOC against multiple threat intelligence sources — MISP, VirusTotal, AbuseIPDB, Shodan.
3. **Hunts** across log data (OpenSearch) for related activity, lateral movement, and persistence indicators.

4. **Correlates** findings to detect campaigns, attribute threat actors, and build an attack timeline.
5. **Reports** by generating a structured incident report and creating a MISP event for community sharing.

### 1.3 Key Technical Requirements

| Requirement | How MAPSEC Addresses It |
|---|---|
| Multi-agent orchestration | LangGraph `StateGraph` with Supervisor pattern, 5 specialized agents |
| Tool use | 30+ LangChain `@tool` functions wrapping MISP, OpenSearch, VirusTotal, AbuseIPDB, Shodan, MITRE ATT&CK |
| Vector database | ChromaDB with 4 collections for RAG-based playbook retrieval, incident correlation, and threat actor matching |
| Structured output | Pydantic models for `SOCState`, `IOC`, `SeverityScore`, `ThreatContext`, `CorrelationResult` |
| Session persistence | LangGraph `MemorySaver` checkpointing with `thread_id` for resumable investigations |

## 2. Theory / Background

### 2.1 Security Operations Center (SOC)

A SOC is a centralized function within an organization that employs people, processes, and technology to continuously monitor and improve an organization's security posture while preventing, detecting, analyzing, and responding to cybersecurity incidents.

**Typical SOC Workflow (Tier 1 Analyst):**

```
Alert Received → Triage → IOC Extraction → Enrichment → Log Analysis
    → Correlation → Incident Report → Remediation → Close
```

Each step historically requires querying different platforms (SIEM, TIP, OSINT, ticketing). This project unifies these into a single automated pipeline.

### 2.2 Multi-Agent Systems

A multi-agent system (MAS) consists of multiple autonomous agents that interact to solve problems beyond the capability of any single agent. In the context of LLM-powered systems:

- **Single-agent**: One LLM with tools handles everything (e.g., a ReAct agent). Works for simple tasks but struggles with complex, multi-step investigations requiring domain specialization.
- **Multi-agent**: Multiple specialized LLM agents, each with their own tool set and system prompt, coordinated by an orchestrator. Each agent is an expert in one domain (triage, enrichment, hunting, etc.).

This project uses the **Supervisor pattern** — a central supervisor agent decides which specialist runs next based on the current investigation state.

### 2.3 LangGraph

LangGraph is a framework for building stateful, multi-step LLM applications as **directed graphs**. Key concepts used in this project:

- **StateGraph**: A graph where nodes are functions (agents) and edges define transitions. The graph operates on a shared `TypedDict` state.
- **Conditional Edges**: The supervisor uses conditional edges to dynamically route to different agents based on completion flags and LLM reasoning.
- **Checkpointing**: `MemorySaver` enables saving graph state at each step, allowing investigation resumption and conversational follow-up.
- **ReAct Agents**: Each specialist agent uses `create_react_agent` — a prebuilt LangGraph pattern where the LLM iteratively reasons, calls tools, observes results, and decides next steps.

### 2.4 Model Context Protocol (MCP)

MCP is an open protocol that standardizes how AI applications connect to external tools and data sources. This project implements two MCP servers:

- **MISP MCP Server** (`misp_mcp_server.py`): Exposes 23 MISP operations as MCP tools via stdio transport.
- **OpenSearch MCP Server** (`mcp-server-opensearch`): Community package for OpenSearch log querying.

In the SOC Agent, MCP server handler functions are imported directly as Python functions and wrapped with LangChain's `@tool` decorator, avoiding the overhead of subprocess-based MCP transport during runtime.

### 2.5 MISP (Malware Information Sharing Platform)

MISP is an open-source threat intelligence platform for sharing, storing, and correlating Indicators of Compromise (IOCs). This project uses MISP as both:

- **Consumer**: Query known-bad IPs, domains, hashes, and associated threat context during enrichment.
- **Producer**: Create new MISP events with discovered IOCs after investigation, feeding intelligence back to the community.

### 2.6 Retrieval-Augmented Generation (RAG)

RAG enhances LLM responses by first retrieving relevant documents from a vector database, then including them in the LLM's context. This project uses RAG for:

- **Playbook retrieval**: Given an alert scenario, semantically search for the most relevant response playbook.
- **Incident correlation**: Find past investigations with similar IOCs, techniques, or attack patterns.
- **Threat actor attribution**: Match observed TTPs to known threat actor profiles.

### 2.7 MITRE ATT&CK Framework

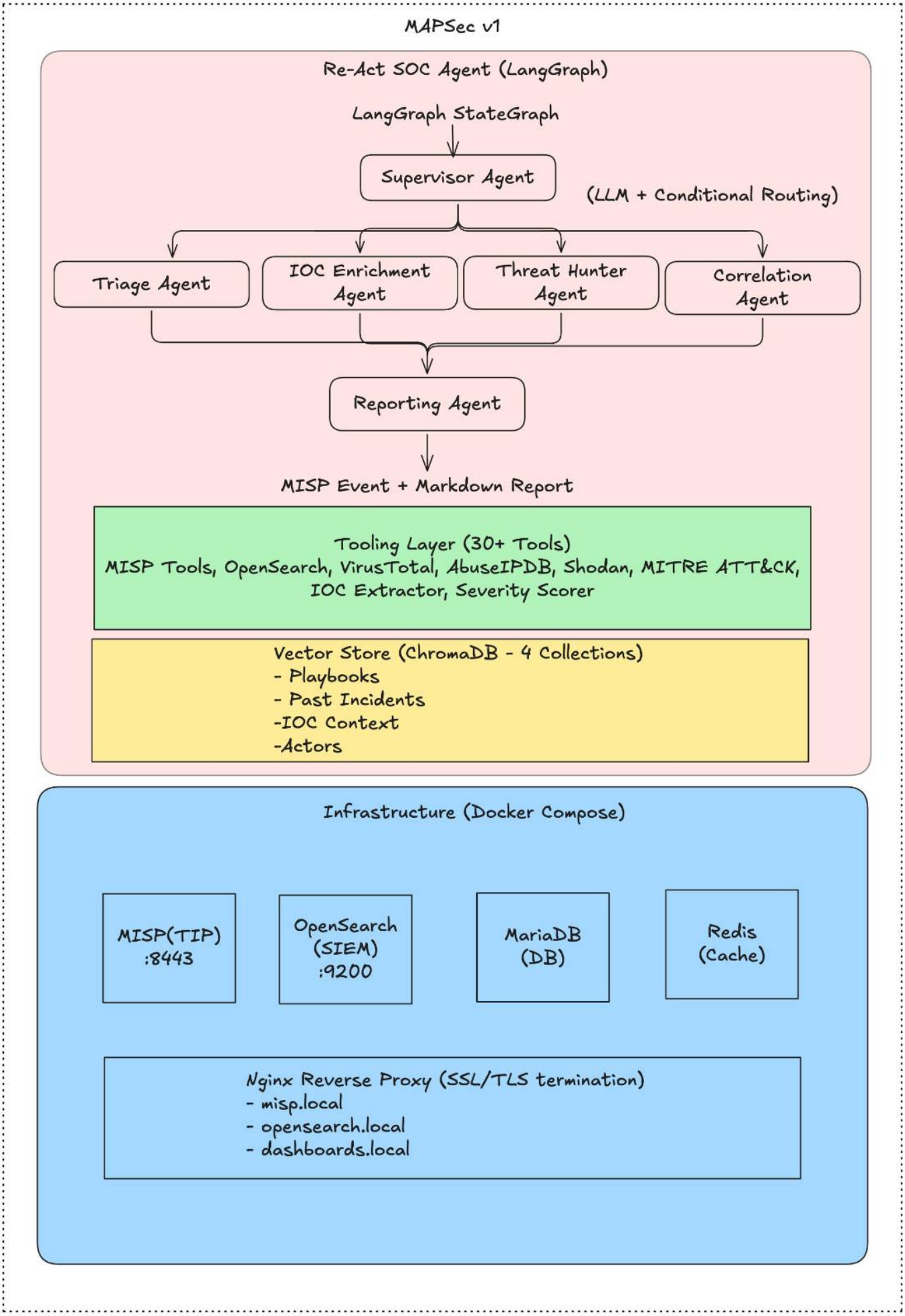
MITRE ATT&CK is a globally-accessible knowledge base of adversary tactics, techniques, and procedures (TTPs) based on real-world observations. This project maps alerts and IOCs to ATT&CK techniques to:

- Classify the attack phase (Initial Access, Lateral Movement, Exfiltration, etc.)
- Score severity based on kill chain position (Exfiltration = 9/10, Discovery = 3/10)
- Provide detection guidance and data source recommendations

---

## 3. System Architecture

### 3.1 High-Level Architecture


MAPSec v1
Re-Act SOC Agent (LangGraph)
LangGraph StateGraph
Supervisor Agent
(LLM + Conditional Routing)
Triage Agent
IOC Enrichment Agent
Threat Hunter Agent
Correlation Agent
Reporting Agent
MISP Event + Markdown Report
Tooling Layer (30+ Tools)
MISP Tools, OpenSearch, VirusTotal, AbuseIPDB, Shodan, MITRE ATT&CK, IOC Extractor, Severity Scorer
Vector Store (ChromaDB - 4 Collections)
- Playbooks
- Past Incidents
-IOC Context
-Actors
Infrastructure (Docker Compose)
MISP(TIP) :8443
OpenSearch (SIEM) :9200
MariaDB (DB)
Redis (Cache)
Nginx Reverse Proxy (SSL/TLS termination)
- misp.local
- opensearch.local
- dashboards.local

### 3.2 Component Breakdown

| Component | Technology | Purpose |
|---|---|---|
| **Orchestration** | LangGraph `StateGraph` | Multi-agent pipeline with conditional routing |
| **LLM** | Claude Sonnet (Anthropic) or GPT-4o (OpenAI) | Agent reasoning, tool selection, report generation |
| **Threat Intelligence** | MISP + MCP Server | IOC lookup, event creation, sighting tracking |
| **Log Analytics** | OpenSearch 2.9.0 | Security log search, aggregations, time-series analysis |
| **Vector Database** | ChromaDB (persistent) | RAG for playbooks, past incidents, threat actor profiles |
| **Embeddings** | `all-MiniLM-L6-v2` (HuggingFace) | Local sentence embeddings, 80MB model, no API cost |
| **External OSINT** | VirusTotal, AbuseIPDB, Shodan | IP/domain/hash reputation, abuse scoring, infrastructure recon |
| **Threat Framework** | MITRE ATT&CK (local STIX) | Technique mapping, kill chain classification |
| **CLI** | Typer + Rich | Terminal interface with live streaming and Markdown rendering |
| **Config** | Pydantic Settings + `.env` | Type-safe configuration with environment variable support |
| **Database** | MariaDB 10.6 | MISP backend storage |
| **Cache** | Redis 6 | MISP job queue and caching |
| **Reverse Proxy** | Nginx | SSL termination, local domain routing |

### 3.3 Directory Structure

```
MAPSEC/
├── soc_agent/                          # Multi-agent SOC system
│   ├── __init__.py
│   ├── __main__.py                     # python -m soc_agent entry
│   ├── main.py                         # Typer CLI (investigate, ingest,
chat)
│   │
│   ├── config/
│   │   └── settings.py                 # Pydantic BaseSettings
│   │
│   ├── graph/
│   │   ├── state.py                    # SOCState TypedDict + Pydantic
```

```
models
|   |   ├── supervisor.py              # Supervisor routing node
|   |   └── graph_builder.py           # StateGraph assembly + compile
|   |
|   ├── agents/
|   |   ├── _llm.py                    # Shared LLM factory
(Anthropic/OpenAI)
|   |   ├── triage.py                  # Alert triage + IOC extraction
|   |   ├── ioc_enrichment.py          # Multi-source IOC enrichment
|   |   ├── threat_hunter.py           # Log-based threat hunting
|   |   ├── correlation.py             # Campaign detection + attribution
|   |   └── reporting.py               # Report generation + MISP event
creation
|   |
|   ├── tools/
|   |   ├── misp_tools.py              # 17 MISP @tool wrappers
|   |   ├── opensearch_tools.py        # OpenSearch search + aggregation
|   |   ├── external_enrichment.py     # VirusTotal, AbuseIPDB, Shodan
|   |   ├── mitre_attack.py            # MITRE ATT&CK STIX lookup +
keyword mapper
|   |   └── utility_tools.py           # IOC regex extractor + severity
scorer
|   |
|   ├── vector_store/
|   |   ├── embeddings.py              # HuggingFace embedding model
|   |   ├── chroma_store.py            # ChromaDB 4-collection setup
|   |   ├── ingestion.py               # Playbook + threat actor
ingestion
|   |   └── retriever.py               # RAG @tool wrappers for semantic
search
|   |
|   ├── playbooks/                     # YAML response playbooks
|   |   ├── ransomware_response.yaml
|   |   ├── phishing_triage.yaml
|   |   ├── lateral_movement.yaml
|   |   └── data_exfiltration.yaml
|   |
|   └── data/
|       ├── mitre_attack_v14.json      # Local MITRE ATT&CK STIX bundle
|       ├── chroma_db/                 # ChromaDB persistent storage
|       └── threat_actor_profiles/     # Seed threat actor JSON files
|           ├── lazarus_group.json
|           └── apt28.json
|
├── misp_mcp_server.py                 # MCP Server — MISP (23 tools,
stdio transport)
├── misp-mcp-server/                   # Alternative FastMCP server
|   └── misp_server.py
|
├── alerts/                            # Sample alert JSON files for demo
|   ├── lateral_movement_alert.json
|   ├── phishing_alert.json
|   └── dns_exfiltration_alert.json
|
```

```
├── scripts/                            # Utility scripts
├── nginx/                              # Nginx reverse proxy
configuration
├── misp_config/                        # MISP database configuration
│
├── docker-compose.yml                  # MISP + OpenSearch + MariaDB +
Redis
├── requirements.txt                    # Python dependencies
├── .env.example                        # Environment variable template
└── Documentation_Report.MD            # This document
```

---

# 4. System Working and Different Use Cases

## 4.1 End-to-End Investigation Flow

The following describes what happens when a security analyst runs:

```
python -m soc_agent investigate alerts/lateral_movement_alert.json --
verbose
```

**Input Alert:**

```
{
  "id": "alert-2026-0314-001",
  "source": "opensearch",
  "timestamp": "2026-03-14T02:15:00Z",
  "description": "Multiple SMB connections from external IP to internal
host with PsExec execution",
  "data": {
    "src_ip": "185.220.101.45",
    "dst_ip": "10.0.0.50",
    "dst_port": 445,
    "process_name": "psexec.exe",
    "connection_count": 847
  }
}
```

**Step-by-step execution:**

```
┌──────────────────────────────────────────────────────────────┐
│  STEP 1: SUPERVISOR (Iteration 1)                            │
│  Checks: triage_complete=False → routes to TRIAGE            │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  STEP 2: TRIAGE AGENT                                        │
```

```
│  1. extract_iocs_from_text → finds 185.220.101.45,          │
│     10.0.0.50, psexec.exe                                   │
│  2. misp_search_ip("185.220.101.45") → MISP hit:            │
│     event_id=42, threat_level=1 (High), tag="apt:lazarus"   │
│  3. map_text_to_mitre_techniques("psexec SMB lateral") →    │
│     T1021.002 (SMB/Windows Admin Shares)                    │
│  4. retrieve_playbook("lateral movement via SMB") →          │
│     lateral_movement.yaml                                   │
│  5. compute_severity_score(misp=1, mitre=8.0) → 8.2         │
│  Output: severity=CRITICAL, 3 IOCs, playbook matched        │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 3: SUPERVISOR (Iteration 2)                           │
│  Checks: triage_complete=True, enrichment_complete=False    │
│  → routes to IOC_ENRICHMENT                                 │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 4: IOC ENRICHMENT AGENT                               │
│  For 185.220.101.45:                                        │
│    1. virustotal_lookup → 67/90 engines: malicious          │
│    2. abuseipdb_lookup → 98% abuse confidence, TOR node     │
│    3. shodan_lookup → ports [80, 443, 9050]                 │
│    4. misp_get_threat_details → Lazarus Group attribution    │
│  Confidence updated: 0.99 (corroborated by 3 sources)       │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 5: SUPERVISOR (Iteration 3)                           │
│  LLM review: enrichment found critical TOR node, default    │
│  routing (threat_hunter) is correct → routes to THREAT_     │
│  HUNTER                                                     │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 6: THREAT HUNTER AGENT                                │
│  1. opensearch_list_indices → discovers available log       │
│     indices (firewall, endpoint, dns)                       │
│  2. opensearch_search_logs("src_ip:185.220.101.45",         │
│     time_range="7d") → 1,247 hits across 3 hosts            │
│  3. opensearch_aggregate(agg_field="destination.ip") →      │
│     10.0.0.50, 10.0.0.67, 10.0.0.89                        │
│  4. opensearch_search_logs("process.name:psexec.exe") →     │
│     43 hits (lateral movement confirmed)                    │
│  5. retrieve_similar_incidents("TOR exit SMB lateral") →    │
│     Past incident from 2024-11-03 (same TOR node)           │
└─────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────┐
│  STEP 7: CORRELATION AGENT                                  │
│  1. 3 hosts compromised in 2h window → active campaign      │
│  2. retrieve_threat_actor_profile("Lazarus North Korea") →  │
│     91% match to known Lazarus Group TTPs                   │
```

```
│  3. Campaign confidence: 0.91, is_campaign=True                │
│  4. Attack timeline built: Initial Access → Lateral            │
│     Movement → C2 Establishment                                │
└────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌────────────────────────────────────────────────────────────────┐
│  STEP 8: REPORTING AGENT                                       │
│  1. misp_add_event("Lazarus Campaign - SMB Lateral             │
│     Movement 2026-03-14", threat_level=1)                      │
│  2. misp_add_attribute(ip-dst: 185.220.101.45)                 │
│  3. misp_add_tag("mitre-attack:T1021.002")                     │
│  4. misp_publish_event(event_id)                               │
│  5. Generates Markdown incident report                         │
│  6. Stores report in ChromaDB for future RAG                   │
└────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌────────────────────────────────────────────────────────────────┐
│  STEP 9: SUPERVISOR (Iteration 6)                              │
│  All *_complete flags = True → routes to END                   │
│  Final report displayed in terminal                            │
└────────────────────────────────────────────────────────────────┘
```

### 4.2 Use Case 1: Lateral Movement Detection

**Scenario**: An external IP establishes multiple SMB connections to an internal workstation, followed by PsExec execution indicating remote code execution.

**Alert file**: `alerts/lateral_movement_alert.json`

**What the system does**:

- Triage identifies the external IP and internal target
- Enrichment confirms the IP is a known TOR exit node associated with Lazarus Group
- Threat hunting reveals 3 compromised internal hosts
- Correlation maps the attack to MITRE T1021.002 and attributes it to a Lazarus campaign
- Reporting creates a MISP event and generates a full incident report with remediation steps

**Value**: An investigation that would take a human analyst 30–45 minutes is completed in 2–3 minutes with full documentation.

### 4.3 Use Case 2: Phishing Triage

**Scenario**: Email gateway detects a macro-enabled Word document attachment from a spoofed sender with credential harvesting URLs.

**Alert file**: `alerts/phishing_alert.json`

**What the system does**:

- Triage extracts sender IP, attachment hashes, and URLs from the email metadata
- Enrichment checks hashes against VirusTotal, URLs against MISP
- Threat hunting searches web proxy logs for employees who clicked the URL

- Correlation checks if this matches a known phishing campaign
- Reporting generates a report with a list of affected users needing password resets

### 4.4 Use Case 3: DNS Exfiltration / Tunneling

**Scenario**: Anomalous DNS query volume with high-entropy subdomain names detected from a development server, indicating potential DNS tunneling for data exfiltration.

**Alert file**: `alerts/dns_exfiltration_alert.json`

**What the system does**:

- Triage identifies the base64-encoded subdomains and the C2 domain
- Enrichment checks the C2 domain against threat intel sources
- Threat hunting searches DNS logs for the exfiltration domain and calculates data volume
- Correlation maps to MITRE T1071.004 (DNS) and T1041 (Exfiltration)
- Reporting quantifies the data exfiltrated and recommends DNS sinkholing

### 4.5 Use Case 4: Interactive Follow-Up

After an investigation completes, analysts can ask follow-up questions:

```
python -m soc_agent chat abc-123-def
> What other hosts did 185.220.101.45 contact in the last 30 days?
> Were any domain admin accounts used during lateral movement?
> Generate a briefing for management about this incident.
```

The chat command resumes the investigation's LangGraph state (via `thread_id` checkpointing), allowing the LLM to access all prior findings.

---

## 5. Pre-requisites and Environment Setup

### 5.1 System Requirements

| Requirement | Minimum | Recommended |
|---|---|---|
| **OS** | macOS, Linux, Windows (WSL2) | macOS or Ubuntu 22.04+ |
| **RAM** | 8 GB | 16 GB |
| **Disk** | 10 GB free | 20 GB free |
| **Python** | 3.11+ | 3.12+ |
| **Docker** | Docker Engine 24+ with Compose v2 | Docker Desktop latest |
| **Network** | Internet access for API calls | Stable broadband |

### 5.2 Required API Keys

| Service | Required? | Free Tier | How to Obtain |
|---|---|---|---|
| **Anthropic** (Claude) | Yes (if using Claude) | Pay-per-use | https://console.anthropic.com/ |
| **OpenAI** (GPT-4o) | Yes (if using OpenAI) | Pay-per-use | https://platform.openai.com/ |
| **VirusTotal** | Optional | 500 lookups/day | https://www.virustotal.com/gui/join-us |
| **AbuseIPDB** | Optional | 1,000 checks/day | https://www.abuseipdb.com/register |
| **Shodan** | Optional | Limited queries | https://account.shodan.io/register |
| **MISP** | Auto-generated | N/A (self-hosted) | Generated during MISP setup |

### 5.3 Installation Steps

#### Step 1: Clone the Repository

```
git clone <repository-url> MAPSEC
cd MAPSEC
```

#### Step 2: Start the Infrastructure Stack

```
# Start MISP + OpenSearch + MariaDB + Redis
docker compose up -d

# Wait for MISP to initialize (first boot takes 3-5 minutes)
docker compose logs -f misp
```

#### Step 3: Configure Local DNS (for MISP)

Add to `/etc/hosts`:

```
127.0.0.1   misp.local
127.0.0.1   opensearch.local
127.0.0.1   opensearch-dashboards.local
```

#### Step 4: (Optional) Start Nginx Reverse Proxy

```
cd nginx
./ssl/gen-certs.sh   # Generate self-signed SSL certificates
```

```
docker compose up -d
cd ..
```

**Step 5: Install Python Dependencies**

```
python -m venv venv
source venv/bin/activate     # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

**Step 6: Configure Environment**

```
cp .env.example .env
# Edit .env with your API keys
```

**Step 7: Retrieve MISP API Key**

```
# Log into MISP web UI at https://misp.local (or https://localhost:8443)
# Default credentials: admin@admin.test / changeme
# Go to: Administration → List Users → View API Key
# Copy the key to .env as MISP_API_KEY
```

**Step 8: Ingest Playbooks and Threat Actor Profiles**

```
python -m soc_agent ingest
```

**Step 9: (Optional) Download MITRE ATT&CK STIX Data**

```
curl -L -o soc_agent/data/mitre_attack_v14.json \
  "https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-attack.json"
```

---

## 6. Configuration Steps

### 6.1 Environment Variables (`.env`)

All configuration is managed through a `.env` file in the project root, loaded by Pydantic's `BaseSettings`:

```
# LLM Provider — choose "anthropic" or "openai"
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4-6
ANTHROPIC_API_KEY=sk-ant-...

# MISP Connection
MISP_URL=https://misp.local
MISP_API_KEY=your-misp-api-key-here
MISP_VERIFY_SSL=false

# OpenSearch Connection
OPENSEARCH_URL=http://localhost:9200
OPENSEARCH_USERNAME=admin
OPENSEARCH_PASSWORD=admin

# External Enrichment (optional — omit to skip)
VIRUSTOTAL_API_KEY=
ABUSEIPDB_API_KEY=
SHODAN_API_KEY=

# Vector Store
CHROMA_DB_PATH=./soc_agent/data/chroma_db
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2

# Agent Execution
MAX_ITERATIONS=15
```

### 6.2 Configuration Class

The `Settings` class in `soc_agent/config/settings.py` uses Pydantic `BaseSettings` for type-safe, validated configuration:

```
class Settings(BaseSettings):
    LLM_PROVIDER: str = "anthropic"     # "anthropic" | "openai"
    LLM_MODEL: str = "claude-sonnet-4-6"
    MISP_URL: str = "https://misp.local"
    MISP_API_KEY: str = ""
    CHROMA_DB_PATH: str = "./soc_agent/data/chroma_db"
    MAX_ITERATIONS: int = 15
    # ... (see config/settings.py for full list)

    class Config:
        env_file = ".env"
        extra = "ignore"
```

**Benefits**: Type validation at startup, default values for optional settings, automatic `.env` loading, and `extra = "ignore"` to tolerate unknown variables.

### 6.3 Docker Compose Configuration

The infrastructure stack (`docker-compose.yml`) defines 5 services:

| Service | Image | Ports | Purpose |
|---|---|---|---|
| `opensearch` | `opensearchproject/opensearch:2.9.0` | 9200, 9600 | Log analytics engine |
| `opensearch-dashboards` | `opensearchproject/opensearch-dashboards:2.9.0` | 5601 | Visualization UI |
| `mariadb` | `mariadb:10.6` | Internal only | MISP database backend |
| `redis` | `redis:6` | Internal only | MISP cache/queue |
| `misp` | `harvarditsecurity/misp:latest` | 8443→443 | Threat intelligence platform |

All services share a Docker bridge network (`mispnet`) for inter-container communication.

---

## 7. Agent Design and Roles

### 7.1 Agent Architecture Overview

Each agent in MAPSEC follows the same structural pattern:

1. **System prompt**: Defines the agent's persona, responsibilities, and expected output format.
2. **Tool set**: A curated list of `@tool`-decorated functions the agent can invoke.
3. **ReAct loop**: Uses `create_react_agent` from LangGraph — the LLM iteratively reasons about which tool to call, observes the result, and decides whether to call another tool or produce a final answer.
4. **Node adapter**: A `*_node()` function that adapts between the ReAct agent's message-based output and the `SOCState` TypedDict, parsing structured JSON from the LLM's response.

```
# Pattern used by all agents:
agent = create_react_agent(llm, tools, prompt=SYSTEM_PROMPT)

def agent_node(state: SOCState) -> dict:
    result = agent.invoke({"messages": [HumanMessage(content=context)]})
    # Parse structured output → update SOCState fields
    return {"field_complete": True, "extracted_data": parsed_data, ...}
```

### 7.2 Triage Agent

| Property | Detail |
|---|---|
| **File** | `soc_agent/agents/triage.py` |

| Property | Detail |
|---|---|
| **Role** | First responder — classifies the alert, extracts IOCs, scores severity |
| **Tools** | `extract_iocs_from_text`, `misp_search_attribute`, `misp_search_ip`, `compute_severity_score`, `map_text_to_mitre_techniques`, `retrieve_playbook` |
| **Output** | `severity`, `extracted_iocs`, `triage_summary`, `retrieved_playbooks` |

**System Prompt Excerpt**:

> "You are a SOC Triage Analyst — the first responder for security alerts. Extract all IOCs, check each against MISP, map to MITRE ATT&CK, retrieve the relevant playbook, and compute a severity score."

**Design Rationale**: The triage agent has access to both IOC extraction tools (regex-based) and MISP lookup tools, but NOT OpenSearch or external enrichment APIs. This enforces the principle of minimal privilege — triage should be fast, not exhaustive.

### 7.3 IOC Enrichment Agent

| Property | Detail |
|---|---|
| **File** | `soc_agent/agents/ioc_enrichment.py` |
| **Role** | Deep-dive enrichment of each IOC from multiple intelligence sources |
| **Tools** | `virustotal_lookup`, `abuseipdb_lookup`, `shodan_lookup`, `misp_get_threat_details`, `misp_search_events`, `misp_add_sighting`, `lookup_mitre_technique`, `retrieve_ioc_context` |
| **Output** | `extracted_iocs` (updated with enrichment data), `enrichment_complete` |

**Key Behavior**: For each IOC, the agent calls up to 4 external APIs in sequence (VirusTotal, AbuseIPDB, Shodan, MISP). It updates the IOC's `confidence` score based on corroborating sources and sets `malicious=True` if VT ratio > 0.3 or AbuseIPDB confidence > 50.

### 7.4 Threat Hunter Agent

| Property | Detail |
|---|---|
| **File** | `soc_agent/agents/threat_hunter.py` |
| **Role** | Proactive threat hunting across log data to find related activity |
| **Tools** | `opensearch_search_logs`, `opensearch_aggregate`, `opensearch_list_indices`, `misp_get_event_galaxies`, `misp_search_events`, `lookup_mitre_technique`, `retrieve_similar_incidents` |
| **Output** | `threat_context`, `opensearch_hits`, `hunt_complete` |

**System Prompt Excerpt**:

> "Formulate targeted OpenSearch queries to find related log activity. Search across multiple time windows: 1h, 24h, 7d. Look for lateral movement patterns, persistence indicators, and C2

> beaconing."

**Design Rationale**: This is the most tool-intensive agent. It generates OpenSearch DSL queries dynamically based on the enriched IOC context, rather than using pre-defined queries.

### 7.5 Correlation Agent

| Property | Detail |
|---|---|
| **File** | `soc_agent/agents/correlation.py` |
| **Role** | Synthesizes all findings to detect campaigns and attribute threat actors |
| **Tools** | `opensearch_aggregate`, `misp_search_events`, `misp_list_galaxy_clusters`, `retrieve_similar_incidents`, `retrieve_threat_actor_profile`, `retrieve_ioc_context` |
| **Output** | `correlation` (CorrelationResult), `retrieved_past_incidents` |

**Key Behavior**: This is the most analytically sophisticated agent. It:

1. Uses OpenSearch aggregations to find shared infrastructure (same ASN, CIDR blocks, user-agents)
2. Semantically searches past incident reports in ChromaDB for similar attack patterns
3. Calculates a `campaign_confidence` score (0.0–1.0)
4. Builds a chronological attack timeline
5. Attributes the attack to a known threat actor if TTP overlap exceeds threshold

### 7.6 Reporting Agent

| Property | Detail |
|---|---|
| **File** | `soc_agent/agents/reporting.py` |
| **Role** | Generates the final incident report and creates a MISP event |
| **Tools** | `misp_add_event`, `misp_add_attribute`, `misp_add_tag`, `misp_publish_event` |
| **Output** | `final_report` (Markdown), `misp_event_id`, `report_complete` |

**Closed-Loop Design**: After generating the report, the agent stores it in ChromaDB's `past_incidents` collection. This means each investigation makes the system smarter — future investigations can find "Have we seen this before?" matches against all past work.

---

## 8. Orchestration and State Management

### 8.1 LangGraph StateGraph

The orchestration layer is implemented as a LangGraph `StateGraph` — a directed graph where:

- **Nodes** are Python functions (agents) that read from and write to a shared state.
- **Edges** define transitions between nodes.
- **Conditional edges** allow the supervisor to dynamically route based on state.

```
# graph_builder.py — Simplified
builder = StateGraph(SOCState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("triage", triage_node)
builder.add_node("ioc_enrichment", ioc_enrichment_node)
builder.add_node("threat_hunter", threat_hunter_node)
builder.add_node("correlation", correlation_node)
builder.add_node("reporting", reporting_node)

builder.add_edge(START, "supervisor")
builder.add_conditional_edges("supervisor", _route_from_supervisor, {...})
for agent in agents:
    builder.add_edge(agent, "supervisor")  # All agents return to
supervisor
```

### 8.2 SOCState: The Shared State Object

The `SOCState` TypedDict is the single source of truth shared across all agents:

```
class SOCState(TypedDict):
    messages: Annotated[list, add_messages]   # Message accumulator
(LangGraph reducer)
    raw_alert: Alert                          # Input (immutable)
    severity: Optional[SeverityScore]         # Set by triage
    triage_complete: bool                     # Completion flag
    extracted_iocs: list[IOC]                 # Set by triage, updated by
enrichment
    enrichment_complete: bool
    threat_context: Optional[ThreatContext]   # Set by threat hunter
    opensearch_hits: list[dict]              # Set by threat hunter
    hunt_complete: bool
    correlation: Optional[CorrelationResult]  # Set by correlation
    correlation_complete: bool
    final_report: Optional[str]              # Set by reporting
    misp_event_id: Optional[str]             # Set by reporting
    report_complete: bool
    iteration_count: int                     # Incremented by supervisor
    next_agent: str                          # Routing decision
    error_log: Annotated[list[str], operator.add]  # Append-only errors
    investigation_id: str                    # UUID for this session
```

**Key Design Decisions**:

1. `Annotated[list, add_messages]`: LangGraph's built-in reducer that appends new messages rather than replacing them, preserving the full conversation history.
2. `Annotated[list[str], operator.add]`: The `error_log` uses Python's `operator.add` as a reducer, ensuring errors from all agents are accumulated.

3. **Completion flags** (`*_complete`): Boolean flags that the supervisor checks for deterministic routing. Each agent sets its flag to `True` when done.
4. **Pydantic sub-models**: `IOC`, `SeverityScore`, `ThreatContext`, and `CorrelationResult` are Pydantic `BaseModel` classes with type validation, defaults, and JSON serialization.

### 8.3 Supervisor Routing Logic

The supervisor implements a **two-tier routing strategy**:

**Tier 1: Deterministic Pipeline** (fast, no LLM call)

```
not triage_complete       → triage
not enrichment_complete   → ioc_enrichment
not hunt_complete         → threat_hunter
not correlation_complete  → correlation
not report_complete       → reporting
all complete              → END
```

**Tier 2: LLM Override** (for edge cases) For non-initial stages, the supervisor optionally consults the LLM to check if the default route should be overridden:

- Enrichment found new IOCs → re-run threat hunter
- Threat hunter found critical evidence → re-run enrichment
- Errors accumulated → force END

**Safety Guard**: `iteration_count > MAX_ITERATIONS (15)` forces termination to prevent infinite loops.

### 8.4 Session Persistence (Checkpointing)

LangGraph's `MemorySaver` (in-memory checkpointer) saves the complete graph state after each node execution:

```
graph = build_soc_graph()  # compiled with MemorySaver

# Investigation 1
config = {"configurable": {"thread_id": "inv-001"}}
graph.invoke(initial_state, config=config)

# Later — resume and ask follow-up questions
state = graph.get_state(config)  # Retrieves full state from checkpoint
```

This enables the `chat` CLI command, which resumes a completed investigation by its thread ID.

---

## 9. Data Sources and Tool Integrations

### 9.1 Tool Inventory

MAPSEC integrates **30+ tools** across 5 categories:

**MISP Tools (17 tools)**

| Tool | Type | Description |
|---|---|---|
| `misp_search_attribute` | Read | Search for any IOC value in MISP |
| `misp_search_ip` | Read | Check if an IP is known-malicious |
| `misp_get_threat_details` | Read | Get full threat context for an indicator |
| `misp_search_events` | Read | Search events with filters (tags, time, type) |
| `misp_get_event` | Read | Get full event details by ID |
| `misp_get_event_indicators` | Read | Get all IOCs grouped by type for an event |
| `misp_list_events` | Read | List recent events |
| `misp_get_malicious_ips` | Read | Get all IPs marked for IDS |
| `misp_get_malicious_domains` | Read | Get all domains marked for IDS |
| `misp_get_malicious_hashes` | Read | Get file hashes (md5/sha1/sha256) |
| `misp_get_event_galaxies` | Read | Get MITRE/malware/actor galaxy data |
| `misp_list_galaxy_clusters` | Read | List galaxy clusters by type |
| `misp_add_event` | Write | Create a new MISP event |
| `misp_add_attribute` | Write | Add an IOC to an event |
| `misp_add_tag` | Write | Tag an event (TLP, MITRE, actor) |
| `misp_add_sighting` | Write | Record an IOC sighting |
| `misp_publish_event` | Write | Publish an event to the community |

**OpenSearch Tools (3 tools)**

| Tool | Description |
|---|---|
| `opensearch_search_logs` | Full-text log search with time range filtering |
| `opensearch_aggregate` | Aggregation queries (top talkers, port distribution) |
| `opensearch_list_indices` | List available log indices and document counts |

**External Enrichment Tools (3 tools)**

| Tool | API | Free Tier Limit |
|---|---|---|
| `virustotal_lookup` | VirusTotal v3 | 500 lookups/day |
| `abuseipdb_lookup` | AbuseIPDB v2 | 1,000 checks/day |

| Tool | API | Free Tier Limit |
|---|---|---|
| `shodan_lookup` | Shodan REST | Basic host info |

**MITRE ATT&CK Tools (2 tools)**

| Tool | Description |
|---|---|
| `lookup_mitre_technique` | Look up a technique by ID (e.g., T1059.001) |
| `map_text_to_mitre_techniques` | Keyword-based mapping from free text to techniques |

**Utility Tools (2 tools)**

| Tool | Description |
|---|---|
| `extract_iocs_from_text` | Regex-based IOC extraction (IPs, domains, hashes, URLs, emails) |
| `compute_severity_score` | Multi-factor weighted severity scoring (0–10) |

**RAG Tools (4 tools)**

| Tool | ChromaDB Collection | Description |
|---|---|---|
| `retrieve_playbook` | `soc_playbooks` | Find relevant response playbook |
| `retrieve_similar_incidents` | `past_incidents` | Find past investigations with similar patterns |
| `retrieve_ioc_context` | `ioc_context` | Find previously enriched IOC profiles |
| `retrieve_threat_actor_profile` | `threat_actors` | Match TTPs to known threat actors |

### 9.2 Vector Database Design (ChromaDB)

ChromaDB runs in-process with persistent on-disk storage — no additional Docker container required.

**Four Collections**:

| Collection | What's Stored | Metadata Fields | Used By |
|---|---|---|---|
| `soc_playbooks` | YAML playbooks converted to text | `playbook_name`, `severity_threshold`, `mitre_techniques` | Triage |
| `past_incidents` | Completed investigation reports | `severity_level`, `threat_actor`, `mitre_techniques`, `date` | Correlation |
| `ioc_context` | Enriched IOC profiles | `ioc_value`, `ioc_type`, `confirmed_malicious` | Correlation |

| Collection | What's Stored | Metadata Fields | Used By |
|---|---|---|---|
| `threat_actors` | Threat actor profile documents | `actor_name`, `origin`, `target_sectors` | Correlation |

**Embedding Model**: `sentence-transformers/all-MiniLM-L6-v2`

- Runs locally on CPU (no API cost)
- 80 MB model size
- 384-dimensional embeddings
- Optimized for semantic similarity of English text

**Ingestion Pipeline**:

```
python -m soc_agent ingest
# Reads YAML playbooks from soc_agent/playbooks/
# Reads JSON profiles from soc_agent/data/threat_actor_profiles/
# Converts to LangChain Documents with metadata
# Embeds and stores in ChromaDB
```

### 9.3 Severity Scoring Algorithm

The severity scorer uses a weighted multi-factor formula:

```
Score = (MISP × 0.25) + (VT × 0.25) + (AbuseIPDB × 0.15)
      + (MITRE × 0.15) + (Asset × 0.10) + (Volume × 0.10)
```

| Factor | Weight | Scale | Source |
|---|---|---|---|
| MISP threat level | 25% | 1=10, 2=6, 3=3, 4=1 | MISP event |
| VirusTotal ratio | 25% | 0.0–1.0 × 10 | VirusTotal API |
| AbuseIPDB confidence | 15% | 0–100 → 0–10 | AbuseIPDB API |
| MITRE tactic severity | 15% | 0–10 (Impact=10, Discovery=3) | MITRE ATT&CK |
| Asset criticality | 10% | 0–10 (from alert metadata) | Alert context |
| IOC volume | 10% | count × 2, max 10 | IOC count |

**Classification**:

- 8.0–10.0 → **CRITICAL**: Immediate incident response required
- 6.0–7.9 → **HIGH**: Escalate to senior analyst
- 3.0–5.9 → **MEDIUM**: Investigate within 4 hours
- 0.0–2.9 → **LOW**: Log and monitor

---

# 10. Prompts and LLM Configuration

## 10.1 LLM Selection

MAPSEC supports two LLM providers, configurable via environment:

| Provider | Model | Temperature | Max Tokens | Use Case |
|---|---|---|---|---|
| **Anthropic** | `claude-sonnet-4-6` | 0 | 4,096 | Default — strong tool use, reasoning |
| **OpenAI** | `gpt-4o` | 0 | 4,096 | Alternative — widely available |

**Temperature = 0**: Deterministic output for consistent, reproducible investigations. SOC analysis should not be creative — it should be accurate.

## 10.2 Prompt Engineering Principles

All agent prompts follow these principles:

1. **Role definition**: Each prompt starts with "You are a [specific SOC role]" to establish persona and expertise boundaries.
2. **Numbered task list**: Explicit, ordered steps the agent must follow.
3. **Tool guidance**: Specific instructions on which tools to use and when.
4. **Structured output**: Required JSON schema at the end of the prompt to ensure parseable output.
5. **Scope boundaries**: Explicit "Do NOT investigate further" to prevent agents from overstepping their role.

## 10.3 Agent Prompt Summaries

**Triage Agent**:

> "You are a SOC Triage Analyst — the first responder for security alerts. Your job: (1) Extract IOCs, (2) Check MISP, (3) Map to MITRE, (4) Retrieve playbook, (5) Score severity. Output structured JSON."

**IOC Enrichment Agent**:

> "You are an IOC Enrichment Specialist. For each IOC: (1) Query VirusTotal, (2) Query AbuseIPDB + Shodan for IPs, (3) Get MISP threat context, (4) Check knowledge base, (5) Update confidence. Be thorough — each IOC should have a complete profile."

**Threat Hunter Agent**:

> "You are a Threat Hunter. (1) List available indices, (2) Formulate OpenSearch queries for IOCs, (3) Search multiple time windows, (4) Look for lateral movement + persistence, (5) Use aggregations for top talkers, (6) Retrieve similar past incidents."

**Correlation Agent**:

> "You are a Threat Correlation Analyst. Answer: (1) Is this a campaign? (2) Have we seen this before? (3) Who is behind this? (4) What's the full timeline? (5) What's the blast radius? Calculate campaign

> confidence 0.0–1.0."

**Reporting Agent**:

> "Generate a comprehensive incident report. Step 1: Create MISP event. Step 2: Add IOC attributes. Step 3: Add tags. Step 4: Write Markdown report with: Executive Summary, Severity, IOC Table, MITRE Mapping, Timeline, Attribution, Recommendations."

**Supervisor**:

> "You are the SOC investigation supervisor. Review status. Respond with JSON: {next_agent, reason}. Override ONLY if: (1) new IOCs need enrichment, (2) critical finding changes hunt scope, (3) errors require termination."

### 10.4 LLM Factory Pattern

The `_llm.py` module implements a singleton factory to avoid creating multiple LLM instances:

```
def get_llm() -> BaseChatModel:
    if settings.LLM_PROVIDER == "anthropic":
        return ChatAnthropic(model=settings.LLM_MODEL, temperature=0)
    else:
        return ChatOpenAI(model=settings.LLM_MODEL, temperature=0)
```

All 5 agents + the supervisor share the same LLM instance, reducing memory usage and API connection overhead.

---

## 11. Evaluation and Performance Metrics

### 11.1 Functional Evaluation Criteria

| Criterion | How to Measure | Target |
|---|---|---|
| **IOC Extraction Accuracy** | Compare extracted IOCs vs. manually identified IOCs in sample alerts | >95% recall |
| **MISP Lookup Success** | % of IOCs successfully checked against MISP | 100% (network permitting) |
| **Enrichment Coverage** | % of IOCs with data from at least 2 sources | >80% for IPs, >60% for others |
| **MITRE Mapping Accuracy** | Compare mapped techniques vs. expert analysis | >85% precision |
| **Report Completeness** | Presence of all 9 required report sections | 100% |
| **MISP Event Quality** | Event has correct threat_level, attributes, and tags | Manual review |

| Criterion | How to Measure | Target |
|---|---|---|
| **End-to-End Latency** | Total time from alert input to report output | <5 minutes |

### 11.2 Agent-Level Metrics

| Agent | Key Metric | How to Track |
|---|---|---|
| Triage | IOCs found, severity score assigned | State: `len(extracted_iocs)`, `severity.score` |
| Enrichment | % IOCs with `malicious` not None | State: count IOCs where `malicious is not None` |
| Threat Hunter | OpenSearch hits returned | State: `len(opensearch_hits)` |
| Correlation | Campaign confidence score | State: `correlation.campaign_confidence` |
| Reporting | MISP event created successfully | State: `misp_event_id is not None` |

### 11.3 System-Level Metrics

| Metric | Source | Notes |
|---|---|---|
| **Iterations used** | `state.iteration_count` | Ideal: 6–8 (one pass per agent + supervisor). Max: 15. |
| **Errors logged** | `state.error_log` | Should be empty for successful investigations |
| **LLM API calls** | Count across all agents | Typically 6–15 calls per investigation |
| **Tool invocations** | Count tool calls in message history | Typically 15–30 per investigation |
| **RAG retrievals** | Count `retrieve_*` tool calls | Typically 3–6 per investigation |

### 11.4 Qualitative Evaluation

The final incident report can be evaluated against these rubric dimensions:

1. **Accuracy**: Are all IOCs correctly identified and classified?
2. **Completeness**: Does the report include all affected systems, techniques, and recommendations?
3. **Actionability**: Can a human analyst act on the recommendations without additional research?
4. **Attribution quality**: Is the threat actor attribution supported by evidence?
5. **Timeline accuracy**: Is the attack timeline chronologically correct with source evidence?

---

## 12. Security and Privacy Considerations

### 12.1 Data Handling

| Data Type | Sensitivity | How MAPSEC Handles It |
|---|---|---|
| Alert data (IOCs, IPs) | Medium | Sent to LLM APIs (encrypted in transit via HTTPS) |
| MISP data | High | Stays within local Docker network; API key required |
| OpenSearch logs | High | Stays within local Docker network; no external transmission |
| Investigation reports | Medium | Stored locally in ChromaDB; optionally shared via MISP |
| API keys | Critical | Stored in `.env` file (never committed to git) |

### 12.2 API Key Security

- All API keys are loaded from `.env` via Pydantic `BaseSettings` — never hardcoded.
- The `.env` file should be added to `.gitignore` (and `.env.example` committed instead).
- External enrichment APIs (VirusTotal, AbuseIPDB, Shodan) only receive IOC values, not full alert context.

### 12.3 LLM Data Exposure

When using cloud-hosted LLMs (Anthropic, OpenAI), the following data is sent:

- Raw alert JSON data
- Extracted IOC values
- MISP query results
- OpenSearch log excerpts
- Agent system prompts

**Mitigation**:

- Use Anthropic's API which does not train on user data by default.
- For highly sensitive environments, switch to a locally-hosted LLM (e.g., Ollama with `llama3.1`).
- The `LLM_PROVIDER` and `LLM_MODEL` settings make this swap straightforward.

### 12.4 Network Security

- MISP runs on a Docker bridge network (`mispnet`), not exposed to the public internet.
- Nginx provides SSL/TLS termination with self-signed certificates for local development.
- OpenSearch has security disabled (`plugins.security.disabled=true`) — appropriate for local-only deployment, not production.

### 12.5 Access Control

- MISP requires authentication (API key) for all operations.
- The SOC Agent runs with a single MISP API key — all events created are attributed to this key's user.
- In production, different agents should use different API keys with scoped permissions (e.g., enrichment gets read-only, reporting gets write access).

---

## 13. Limitations and Future Scope

### 13.1 Current Limitations

| Limitation | Impact | Potential Solution |
|---|---|---|
| **LLM dependency** | Requires API access; costs per investigation | Support local LLMs (Ollama, vLLM) |
| **No real-time alerting** | System is reactive (analyst triggers investigation) | Add OpenSearch alerting integration or Kafka consumer |
| **Single-analyst mode** | No multi-user support or RBAC | Add FastAPI layer with authentication |
| **English-only** | System prompts and NLP are English-centric | Multilingual prompts and embedding models |
| **No automated remediation** | Reports actions but doesn't execute them | Integrate with EDR/SOAR APIs for automated isolation |
| **OpenSearch schema dependency** | Log queries assume specific field names | Add schema discovery and adaptive query generation |
| **Limited playbook library** | Only 4 playbooks currently | Expand with industry-standard playbooks (NIST, SANS) |
| **No feedback loop** | No mechanism for analysts to rate or correct findings | Add feedback ingestion to fine-tune scoring and prompts |
| **Embedding model limitations** | `all-MiniLM-L6-v2` is general-purpose, not security-domain | Fine-tune on CTI corpus or use `SecBERT` embeddings |

### 13.2 Future Scope

**Short-Term Enhancements**

1. **FastAPI REST Interface**: Expose the investigation pipeline as an API for integration with ticketing systems (Jira, ServiceNow).
2. **Persistent Checkpointing**: Replace `MemorySaver` with `SqliteSaver` for cross-session persistence that survives process restarts.
3. **Streaming UI**: Build a Streamlit or Gradio frontend for real-time visualization of the agent pipeline.
4. **More Playbooks**: Add playbooks for insider threat, supply chain compromise, DDoS, and cloud-specific incidents.

**Medium-Term Enhancements**

5. **SOAR Integration**: Connect to Palo Alto XSOAR, Splunk SOAR, or TheHive for automated containment actions (host isolation, IP blocking).
6. **Real-Time Alert Ingestion**: Add a Kafka/Redis consumer that triggers investigations automatically when high-severity alerts arrive.
7. **Multi-Tenancy**: Support multiple organizations with tenant-isolated MISP instances and ChromaDB collections.

8. **Human-in-the-Loop**: Add approval gates in the LangGraph pipeline where critical actions (e.g., host isolation, MISP publication) require human confirmation.

**Long-Term Vision**

9. **Federated Threat Intelligence**: Cross-organization IOC sharing where MAPSEC instances at different SOCs collaboratively investigate campaigns.
10. **Reinforcement Learning**: Train the supervisor's routing decisions on historical investigation outcomes to optimize agent sequencing.
11. **Autonomous SOC Tier 1**: Fully automated alert triage and response for known playbook scenarios, with human escalation for novel threats.

---

## 14. Conclusion

MAPSEC demonstrates that multi-agent AI systems can fundamentally transform Security Operations Center workflows. By decomposing the SOC analyst's investigation process into five specialized agents — Triage, IOC Enrichment, Threat Hunting, Correlation, and Reporting — orchestrated through a LangGraph supervisor pattern, the system achieves:

1. **Automation of repetitive SOC tasks**: IOC extraction, MISP lookups, log querying, and report generation are fully automated, reducing investigation time from 30–60 minutes to 2–5 minutes.

2. **Consistent, auditable investigations**: Every investigation follows the same pipeline with the same tools, producing structured reports with evidence citations. The `SOCState` TypedDict provides a complete audit trail.

3. **Intelligence amplification through RAG**: The ChromaDB vector store enables the system to learn from past investigations — each completed case makes future correlation and attribution more accurate.

4. **Closed-loop intelligence sharing**: The Reporting Agent creates MISP events with IOCs, tags, and MITRE mappings, feeding intelligence back to the community and enriching future triage lookups.

5. **Extensible architecture**: New tools, agents, and playbooks can be added without modifying the core graph structure. The MCP server pattern means new data sources require only a new `@tool` wrapper.

The project integrates several advanced concepts — multi-agent orchestration (LangGraph), tool use (30+ LangChain tools), vector databases (ChromaDB with 4 collections), retrieval-augmented generation, structured state management (Pydantic + TypedDict), and real threat intelligence infrastructure (MISP + OpenSearch) — into a cohesive system that addresses a genuine operational need in cybersecurity.

---

## 15. Appendices

### Appendix A: Complete Tool Reference

#### A.1 MISP Tools (`soc_agent/tools/misp_tools.py`)

```
# Read tools
misp_search_attribute(value: str, attribute_type: str = "") -> str
misp_search_ip(ip: str) -> str
misp_get_threat_details(indicator: str) -> str
misp_search_events(tags, attribute_type, value, last, limit) -> str
misp_get_event(event_id: str) -> str
misp_get_event_indicators(event_id: str) -> str
misp_list_events(limit: int = 10) -> str
misp_get_malicious_ips() -> str
misp_get_malicious_domains() -> str
misp_get_malicious_hashes(hash_type: str = "all") -> str
misp_get_event_galaxies(event_id: str) -> str
misp_list_galaxy_clusters(galaxy: str = "", limit: int = 50) -> str

# Write tools
misp_add_event(info, threat_level_id, analysis, distribution) -> str
misp_add_attribute(event_id, attribute_type, value, category, to_ids,
comment) -> str
misp_add_tag(event_id: str, tag: str) -> str
misp_add_sighting(attribute_id: str, sighting_type: int = 0) -> str
misp_publish_event(event_id: str) -> str
```

#### A.2 OpenSearch Tools (`soc_agent/tools/opensearch_tools.py`)

```
opensearch_search_logs(query_string, index="*", time_range="24h", size=50)
-> str
opensearch_aggregate(query_string, agg_field, index="*", time_range="24h")
-> str
opensearch_list_indices() -> str
```

#### A.3 External Enrichment (`soc_agent/tools/external_enrichment.py`)

```
virustotal_lookup(ioc: str, ioc_type: str = "ip") -> str
abuseipdb_lookup(ip: str) -> str
shodan_lookup(ip: str) -> str
```

#### A.4 MITRE ATT&CK (`soc_agent/tools/mitre_attack.py`)

```
lookup_mitre_technique(technique_id: str) -> str
map_text_to_mitre_techniques(text: str) -> str
```

#### A.5 Utility Tools (`soc_agent/tools/utility_tools.py`)

```
extract_iocs_from_text(text: str) -> str
compute_severity_score(misp_threat_level, virustotal_malicious_ratio, ...)
-> str
```

#### A.6 RAG Tools (`soc_agent/vector_store/retriever.py`)

```
retrieve_playbook(scenario: str) -> str
retrieve_similar_incidents(query: str, k: int = 3) -> str
retrieve_ioc_context(query: str, k: int = 5) -> str
retrieve_threat_actor_profile(description: str) -> str
```

### Appendix B: State Schema Reference

```
# Pydantic models
Alert(alert_id, source, raw_data, timestamp, description)
IOC(value, ioc_type, source, confidence, malicious, enrichment_data,
misp_hit, mitre_techniques)
SeverityScore(score, level, factors, recommended_action)
ThreatContext(mitre_techniques, threat_actors, malware_families,
kill_chain_phase, attack_summary)
CorrelationResult(correlated_alert_ids, pattern_description, is_campaign,
campaign_confidence,
                  threat_actor_attribution, attack_timeline,
similar_past_incidents)

# SOCState fields
messages                    # Annotated[list, add_messages]
raw_alert                   # Alert
severity                    # Optional[SeverityScore]
triage_complete             # bool
triage_summary              # str
extracted_iocs              # list[IOC]
enrichment_complete         # bool
threat_context              # Optional[ThreatContext]
opensearch_hits             # list[dict]
hunt_complete               # bool
correlation                 # Optional[CorrelationResult]
correlation_complete        # bool
retrieved_playbooks         # list[dict]
retrieved_past_incidents    # list[dict]
final_report                # Optional[str]
misp_event_id               # Optional[str]
report_complete             # bool
iteration_count             # int
next_agent                  # str
error_log                   # Annotated[list[str], operator.add]
investigation_id            # str
```

### Appendix C: Sample Alert Format

```
{
  "id": "alert-2026-0314-001",
  "source": "opensearch",
  "timestamp": "2026-03-14T02:15:00Z",
  "description": "Human-readable description of the alert",
  "data": {
    "src_ip": "185.220.101.45",
    "dst_ip": "10.0.0.50",
    "dst_port": 445,
    "process_name": "psexec.exe",
    "connection_count": 847,
    "alert_rule": "High volume SMB connections from external source",
    "additional_context": {
      "dst_hostname": "WORKSTATION-050",
      "network_zone": "corporate-lan"
    }
  }
}
```

**Required fields**: `id`, `data` **Optional fields**: `source`, `timestamp`, `description` The `data` field is schema-flexible — the triage agent extracts IOCs regardless of field naming.

### Appendix D: CLI Usage Reference

```
# Investigate an alert
python -m soc_agent investigate <alert.json> [--verbose] [--thread-id
<id>]

# Ingest playbooks and threat actor profiles into ChromaDB
python -m soc_agent ingest

# Resume an investigation interactively
python -m soc_agent chat <thread-id>
```

### Appendix E: Playbook YAML Schema

```
name: "Playbook Name"
trigger_keywords:
  - keyword1
  - keyword2
severity_threshold: "HIGH"          # LOW | MEDIUM | HIGH | CRITICAL
mitre_techniques:
  - "T1059.001"
  - "T1021.002"
steps:
  - id: 1
```

```
    action: "action_name"
    description: "What this step does"
    automated: true                  # true = agent handles it; false =
human action
    agent: "triage"                  # Which agent runs this step (if
automated)
    soc_instruction: "..."           # Human instructions (if not
automated)
    opensearch_queries:              # Pre-defined queries (optional)
      - "query string 1"
      - "query string 2"
```

Appendix F: Dependency List

| Package | Version | Purpose |
|---|---|---|
| langgraph | >=0.2.0 | Multi-agent graph orchestration |
| langchain | >=0.3.0 | LLM framework core |
| langchain-core | >=0.3.0 | Base abstractions (tools, messages) |
| langchain-anthropic | >=0.3.0 | Claude LLM integration |
| langchain-openai | >=0.3.0 | OpenAI LLM integration |
| langchain-mcp-adapters | >=0.1.0 | MCP-to-LangChain bridge |
| chromadb | >=0.5.0 | Vector database |
| langchain-chroma | >=0.1.0 | LangChain ChromaDB integration |
| langchain-huggingface | >=0.1.0 | HuggingFace embeddings |
| sentence-transformers | >=3.0.0 | Local embedding models |
| mcp | 1.26.0 | Model Context Protocol |
| pymisp | 2.5.32.3 | MISP Python library |
| opensearch-py | 3.1.0 | OpenSearch client |
| pydantic | 2.12.5 | Data validation |
| pydantic-settings | 2.12.0 | Settings management |
| typer | >=0.12.0 | CLI framework |
| rich | >=13.0.0 | Terminal formatting |
| httpx | 0.28.1 | HTTP client for external APIs |
| pyyaml | >=6.0 | YAML playbook parsing |